

CSA1208 – COMPUTER ARCHITECTURE

ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

1. Real-Time Embedded I/O Communication Mechanism

1. Problem Understanding

The system has to communicate with multiple sensors in real time. The main requirements are CPU utilization below 15%, sensor interrupts every 1 ms, vectored interrupt handling, and low-latency I/O.

2. Constraint Analysis

- CPU utilization must remain below 15%.
- Multiple sensors generate interrupts every 1 ms.
- Vectored interrupt handling is required.
- The I/O method should provide optimal latency.

3. Pseudocode / Solution

```
START
Initialize CPU
Initialize interrupt controller
Initialize all sensors

FOR each sensor
    Assign interrupt vector
    Configure sensor registers
    Enable sensor interrupt
END FOR

Set CPU utilization limit = 15%

WHILE system is running
    IF sensor interrupt occurs THEN
        Identify sensor using interrupt vector
        Save CPU context
        Execute corresponding ISR
        Read sensor data
        Store data in memory
        Clear interrupt flag
        Restore CPU context
    END IF

    IF CPU utilization > 15% THEN
        Reduce unnecessary processing
        Use DMA for bulk data transfer
    END IF
END WHILE

STOP
```

4. Application of Concepts

Vectored interrupts allow the processor to identify the required service routine directly. Memory-mapped I/O gives efficient access to sensor registers. DMA can be used when sensor data blocks are large, reducing CPU work.

5. Justification

The design keeps interrupt service routines short and uses DMA for bulk transfers. This reduces processor overhead while maintaining fast response to the sensors.

2. NVMe Over PCIe I/O Subsystem

1. Problem Understanding

The storage subsystem must provide throughput above 3 GB/s, use NVMe over PCIe, avoid CPU handling of bulk data, and use DMA with interrupt notification.

2. Constraint Analysis

- Storage throughput must be greater than 3 GB/s.
- NVMe over PCIe must be used.
- The CPU should not handle bulk data transfer.
- DMA must be integrated with interrupt notification.

3. Pseudocode / Solution

```
START
Initialize PCIe interface
Initialize NVMe controller
Initialize DMA controller
Initialize interrupt controller

Configure NVMe queues
Configure DMA source and destination

WHILE storage request exists
    Receive read/write request

    IF write request THEN
        Set source = Memory
        Set destination = NVMe
    ELSE
        Set source = NVMe
        Set destination = Memory
    END IF

    Configure DMA transfer
    Start NVMe command
    DMA transfers bulk data
    CPU performs other tasks

    WHEN transfer completes
        DMA generates interrupt
        CPU checks transfer status

        IF successful THEN
            Notify completion
        ELSE
            Report error
        END IF
    END WHILE
```

STOP

4. Application of Concepts

PCIe provides the high-speed link, NVMe manages storage commands, and DMA moves bulk data without requiring the CPU to copy each block. An interrupt is generated when the transfer is complete.

5. Justification

NVMe over PCIe with DMA is suitable for the high-throughput requirement because bulk data movement is handled by DMA and the CPU is notified only when an operation needs attention.

3. Multi-Device USB and Thunderbolt Communication

1. Problem Understanding

The architecture contains six USB peripherals and two high-speed Thunderbolt devices. It must avoid interrupt starvation, support hardware and software interrupts, and provide fair bus arbitration.

2. Constraint Analysis

- 6 USB peripherals must be supported.
- 2 high-speed Thunderbolt devices must be supported.
- Interrupt starvation must be avoided.
- Hardware and software interrupts must be used.
- Bus access must be fair.

3. Pseudocode / Solution

```
START
Initialize USB controller
Initialize Thunderbolt controller
Initialize interrupt controller

Connect 6 USB devices
Connect 2 Thunderbolt devices

Assign priority to each device
Initialize round-robin queue

WHILE system is running
    Check hardware interrupt requests

    IF high-priority interrupt occurs THEN
        Service high-priority device
    ELSE
        Select next device using round-robin arbitration
    END IF

    Grant bus access

    IF device = USB THEN
        Perform USB communication
    ELSE IF device = Thunderbolt THEN
        Perform Thunderbolt communication
```

END IF

Release bus

IF background task requires service THEN

 Generate software interrupt

 Execute software interrupt routine

END IF

 Update device queue

END WHILE

STOP

4. Application of Concepts

A priority mechanism handles urgent requests, while round-robin arbitration gives waiting devices a fair opportunity to communicate. Hardware interrupts are used for device events and software interrupts for background work.

5. Justification

Combining priority handling with round-robin scheduling prevents a low-priority device from being blocked indefinitely while still allowing urgent device events to be serviced quickly.

4. High-Speed Data Acquisition System

1. Problem Understanding

The data acquisition system receives a continuous data stream at 500 MB/s. CPU intervention should be minimal. DMA must be compared with interrupt-driven I/O, and device registers must use memory-mapped I/O.

2. Constraint Analysis

- Continuous data stream = 500 MB/s.
- CPU intervention must be minimal.
- DMA transfer must be compared with interrupt-driven I/O.
- Device registers must use memory-mapped I/O.

3. Pseudocode / Solution

START

 Initialize data acquisition device

 Initialize DMA controller

 Initialize memory

 Map device registers into memory

 Set transfer rate = 500 MB/s

 Set DMA source = Device

 Set DMA destination = Memory

 Start DMA transfer

WHILE data acquisition is active

 Device generates data

 DMA transfers data directly to memory

CPU performs other processing

IF DMA block transfer completes THEN

 Generate interrupt

 CPU checks transfer status

 Process completed data block

 Configure next DMA block

END IF

END WHILE

STOP

4. Application of Concepts

DMA is better suited to a continuous 500 MB/s stream because the CPU does not need to handle each transfer. Interrupt-driven I/O would require much more CPU attention when data arrives continuously. Memory-mapped I/O is used for device registers.

5. Justification

DMA minimizes CPU intervention and is therefore the preferred transfer method for the continuous high-speed stream. Interrupts are used at block completion rather than for every data item.

5. Nested Vectored Interrupt Mechanism

1. Problem Understanding

The interrupt mechanism must support nested interrupts, keep maximum interrupt latency below 10 μ s, use vectored interrupts for high-priority devices, and use software interrupts for background tasks.

2. Constraint Analysis

- Nested interrupts must be supported.
- Maximum interrupt latency must be below 10 μ s.
- High-priority devices must use vectored interrupts.
- Background tasks must use software interrupts.

3. Pseudocode / Solution

START

 Initialize interrupt controller

 Create interrupt vector table

 Assign priority levels

 Enable nested interrupts

WHILE system is running

 IF interrupt occurs THEN

 Identify interrupt source

 Get interrupt vector

 Check priority

 IF new interrupt has higher priority THEN

 Save current CPU context

 Jump to corresponding ISR

 Execute ISR quickly

 Clear interrupt flag

```
    Restore CPU context
ELSE
    Store interrupt in pending queue
END IF
END IF

IF background task requires processing THEN
    Generate software interrupt
    Execute software interrupt routine
END IF

Monitor interrupt latency

IF latency  $\geq 10 \mu\text{s}$  THEN
    Optimize ISR
    Reduce unnecessary processing
END IF
END WHILE

STOP
```

4. Application of Concepts

Vectored interrupts allow rapid selection of the correct ISR. Nested interrupts let a higher-priority interrupt preempt a lower-priority ISR. Short ISRs help keep interrupt latency within the required limit, while software interrupts handle background tasks.

5. Justification

The design gives priority to urgent hardware events and keeps the ISR short. This supports nested processing and is intended to maintain the maximum interrupt latency below $10 \mu\text{s}$.